LAPORAN PRAKTIKUM ALGORITMA DAN STRUKTUR DASAR

JOBSHEET 5 : SORTING

Oleh :

SALSABILA KEISHA AYU SETIADI

254107020048

Kelas TI-1C

PROGRAM STUDI D-IV TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI MALANG

A. Praktikum 1 : Mengimplementasikan Sorting Menggunakan Object

1. Kode Program *class* Sorting26 :

```java
package Minggu5;

public class Sorting26 {
    int [] data;
    int jumData;

    Sorting26 (int Data[], int jmlDat) {
    jumData = jmlDat;
    data = new int[jmlDat];
        for (int i=0; i < jumData; i++) {
            data[i] = Data[i];
        }
    }

    void bubbleSort() {
        int temp=0;
        for (int i=0; i<jumData-1; i++) {
            for (int j=1; j<jumData-i; j++) {
                if (data[j-1]>data[j]) {
                    temp=data[j];
                    data[j] = data[j-1];
                    data[j-1]=temp;
                }
            }
        }
    }

    void SelectionSort() {
        for (int i = 0; i<jumData-1; i++) {
            int min = i;
            for (int j=i+1; j<jumData; j++) {
                if (data[j]<data[min]) {
                    min = j;
                }
            }
            int temp=data[i];
            data[i]=data[min];
            data[min] = temp;
        }
    }

    void tampil() {
        for (int i=0; i<jumData; i++) {
            System.out.print(data[i] + " ");
        }
        System.out.println();
    }
}
```

- Penambahan method InsertionSort()

```java
void InsertionSort() {
    for (int i = 0; i<=data.length-1; i++) {
        int temp=data[i];
        int j=i-1;

        while (j>=0 && data[j]>temp) {
            data[j+1] = data[j];
            j--;
        }
        data[j+1] = temp;
    }
}
```

2. Kode Program *Main* SortingMain26 :

```
Minggu5 > SortingMain26.java > SortingMain26 > main(String[])
package Minggu5;

public class SortingMain26 {
    Run | Debug
    public static void main(String[] args) {

        int a[] = {20, 10, 2, 7, 12};
        int b[] = {30, 20, 2, 8, 14};
        int c[] = {40, 10, 4, 9, 3};

        Sorting26 dataurut1 = new Sorting26(a, a.length);
        Sorting26 dataurut2 = new Sorting26(b, b.length);
        Sorting26 dataurut3 = new Sorting26(c, c.length);


        System.out.println(x: "Data awal 1");
        dataurut1.tampil();
        dataurut1.bubbleSort();
        System.out.println(x: "Data sudah diurutkan dengan Bubble Sort (ASC)");
        dataurut1.tampil();

        System.out.println(x: "Data awal 2");
        dataurut2.tampil();
        dataurut2.SelectionSort();
        System.out.println(x: "Data sudah diurutkan dengan Selection Sort (ASC)");
        dataurut2.tampil();

        System.out.println(x: "Data awal 3");
        dataurut3.tampil();
        dataurut3.SelectionSort();
        System.out.println(x: "Data sudah diurutkan dengan Insertion Sort (ASC)");
        dataurut3.tampil();

    }
}
```

3. Output Kode program Sorting26 :

```
Data awal 1
20 10 2 7 12
Data sudah diurutkan dengan Bubble Sort (ASC)
2 7 10 12 20
PS C:\Users\ASUS\OneDrive\Documents\kei's\Praktikum ASD\PraktikumASD>
```

```
Data awal 2
30 20 2 8 14
Data sudah diurutkan dengan Selection Sort (ASC)
2 8 14 20 30
PS C:\Users\ASUS\OneDrive\Documents\kei's\Praktikum ASD\PraktikumASD>
```

```
Data awal 3
40 10 4 9 3
Data sudah diurutkan dengan Insertion Sort (ASC)
3 4 9 10 40
```

4. Jawaban dari pertanyaan :
   a. Potongan kode tersebut menjelaskan bahwa jika Data (j-1) lebih besar daripada Data j itu sendiri maka nilai dari Data (j-1) akan digeser ke tempat nilai Data j. Dalam kode tersebutm, nilai dari data j akan ditempatkan ke variable temp, lalu nilai dari data (j-1) akan ditempatkan di data j dan nilai dari data j-1 akan diganti dengan nilai temp tadi.

b. Kode program yang merupakan algoritma pencarian nilai minimum adalah :

```java
for (int j=i+1; j<jumData; j++) {
    if (data[j]<data[min]) {
        min = j;
    }
}
```

Dalam kode tersebut terlihat bahwa perulangan terjadi jika nilai data j lebih kecil daripada nilai data (min), maka nilai min akan sama dengan j. Selama looping terjadi akan mengecek, nilai mana yang lebih kecil daripada nilai data (min), jika ada, maka nilai variabel min akan berganti sesuai dengan variable j tersebut. Hal ini menghasilkan nilai minimum karena selama looping tersebut akan terus membandingkan dengan nilai awal (data min) apakah lebih besar atau lebih kecil. Jika ketemu yang lebih kecil, maka nilai min sendir akan berubah ke nilai j yang mana merupakan letak nilai terkecil tersebut.

c. Pada Insertion Sort perulangan tersebut berguna untuk mengecek apakah nilai sebelah kanan (nilai i) itu lebih besar daripada nilai data j. Jika nilai data i lebih besar, maka perulangan diluar akan tetap berlanjut ke angka sebelahnya. Tetapi jika tidak, maka akan melakukan kondisi di dalam while tersebut.

d. Perintah tersebut bertujuan untuk menjadikan nilai data j menjadi nilai yang berada di sebelahnya (nilai data j+1). Perintah ini terjadi karena nilai J lebih besar daripada nilai data i/temp.

B. Praktikum 2 : Sorting Menggunakan Array of Object

1. Kode Program Main Mahasiswa26 :

Minggu5 > Mahasiswa26.java > ...

```java
package Minggu5;

public class Mahasiswa26 {
    String nim;
    String nama;
    String kelas;
    double ipk;

    public Mahasiswa26() {
    }

    public Mahasiswa26(String nm, String name, String kls, double ip) {
        nim = nm;
        nama = name;
        kelas = kls;
        ipk = ip;
    }

    void tampilInformasi() {
        System.out.println("Nama : " + nama);
        System.out.println("NIM : " + nim);
        System.out.println("Kelas : " + kelas);
        System.out.println("IPK : " + ipk);
    }
}
```

2. Kode Program Class MahasiswaBerprestasi26 :

```java
package Minggu5;

public class MahasiswaBerprestasi26 {
    Mahasiswa26 [] listMhs = new Mahasiswa26 [5];
    int idx;

    void tambah(Mahasiswa26 mhs) {
        if (idx<listMhs.length) {
            listMhs[idx]=mhs;
            idx++;
        } else {
            System.out.println(x: "data sudah penuh");
        }
    }

    void tampil() {
        for (Mahasiswa26 mhs:listMhs) {
            mhs.tampilInformasi();
            System.out.println(x: "----------------------------");
        }
    }

    void bubbleSort() {
        for (int i=0; i<listMhs.length-1; i++) {
            for (int j=1; j<listMhs.length; j++) {
                if (listMhs[j].ipk>listMhs[j-1].ipk) {
                    Mahasiswa26 tmp = listMhs[j];
                    listMhs[j] = listMhs[j-1];
                    listMhs[j-1] = tmp;
                }
            }
        }
    }
}
```

3. Kode Program Main MahasiswaDemo26 :

```java
package Minggu5;

public class MahasiswaDemo26 {
    Run | Debug
    public static void main(String[] args) {
        MahasiswaBerprestasi26 list = new MahasiswaBerprestasi26();
        Mahasiswa26 m1 = new Mahasiswa26(nm: "123", name: "Zidan", kls: "2A", ip: 3.2);
        Mahasiswa26 m2 = new Mahasiswa26(nm: "124", name: "Ayu", kls: "2A", ip: 3.5);
        Mahasiswa26 m3 = new Mahasiswa26(nm: "125", name: "Sofi", kls: "2A", ip: 3.1);
        Mahasiswa26 m4 = new Mahasiswa26(nm: "126", name: "Sita", kls: "2A", ip: 3.9);
        Mahasiswa26 m5 = new Mahasiswa26(nm: "127", name: "Miki", kls: "2A", ip: 3.7);

        list.tambah(m1);
        list.tambah(m2);
        list.tambah(m3);
        list.tambah(m4);
        list.tambah(m5);

        System.out.println(x: "Data mahasiswa sebelum sorting: ");
        list.tampil();

        System.out.println(x: "Data mahasiswa setelah sorting berdasarkan IPK (DESC) : ");
        list.bubbleSort();
        list.tampil();
    }
}
```

4. Output Kode Program :

```
Data mahasiswa sebelum sorting:
Nama : Zidan
NIM : 123
Kelas : 2A
IPK : 3.2
-----------------------------
Nama : Ayu
NIM : 124
Kelas : 2A
IPK : 3.5
-----------------------------
Nama : Sofi
NIM : 125
Kelas : 2A
IPK : 3.1
-----------------------------
Nama : Sita
NIM : 126
Kelas : 2A
IPK : 3.9
-----------------------------
Nama : Miki
NIM : 127
Kelas : 2A
IPK : 3.7
-----------------------------
Data mahasiswa setelah sorting berdasarkan IPK (DESC) :
Nama : Sita
NIM : 126
Kelas : 2A
IPK : 3.9
-----------------------------
Nama : Miki
NIM : 127
Kelas : 2A
IPK : 3.7
-----------------------------
Nama : Ayu
NIM : 124
Kelas : 2A
IPK : 3.5
-----------------------------
Nama : Zidan
NIM : 123
Kelas : 2A
IPK : 3.2
-----------------------------
Nama : Sofi
NIM : 125
Kelas : 2A
IPK : 3.1
-----------------------------
PS C:\Users\ASUS\OneDrive\Documents\kei's\Praktikum ASD\PraktikumASD>
```

5. Jawaban dari pertanyaan :
   a. Karena, pada dasarnya bubble sort adalah metode mengurutkan dengan cara membandingkan kedua angka secara berulang. Pada perulangan i memiliki kondisi i<listMhs.length-1 karena perulangan sudah pasti selesai sebelum indeks terakhir. Jika sudah sampai di satu indeks sebelum indeeks paling terakhir, maka otomatis indeks terakhir adalah angka yang terbesar/terkecil. Sehingga perulangan i tidak perlu dilakukan kembali.
   b. Karena bubble sort selalu memulai perhitungan di satu indeks setelah indeks sebelumnya dilakukan. Jika perulangan pertama sudah selesai, maka saat akan melakukan perulangan setelahnya akan dilakukan di indeks setelah indeks pertama. Karena dilakukan di indeks setelahnya, sementara jumlah indeks pada array tetap sama, maka kondisi perulangan kedua harus kurang dari jumlah perulangan pertama. Semisal perulangan pertama baru dilakukan sekali, maka perulangan didalamnya harus dilakukan sebanyak isi array dikurangi perulangan yang sudah terjadi diluar (1).

c. Jika banyak data listMhs adalah 50, maka otomatis i akan berjalan sebanyak 49 kali. Seperti kondisi yang sudah saya jelaskan di nomor sebelumnya. Tahap bubble sort yang ditempuh sesuai dengan iterasi i itu sendiri, sehingga banyaknya tahapan bubble sort yang ditempuh adalah 49 kali juga.

6. Modifikasi menjadi input dinamis :

```
public class MahasiswaDemo26 {
    Run | Debug
    public static void main(String[] args) {

        Scanner sc2 = new Scanner(System.in);
        MahasiswaBerprestasi26 list = new MahasiswaBerprestasi26();

        String nama, kelas, nim;
        double ipk;

        // Setekah dimodifikasi
        for (int i = 0; i < 5; i++) {
            System.out.println("Masukkan Data Mahasiswa ke-" + (i+1));
            System.out.print(s: "NIM    : ");
            nim = sc2.nextLine();
            System.out.print(s: "Nama   : ");
            nama = sc2.nextLine();
            System.out.print(s: "Kelas  : ");
            kelas = sc2.nextLine();
            System.out.print(s: "IPK    : ");
            ipk = sc2.nextDouble();
            sc2.nextLine();

            Mahasiswa26 m = new Mahasiswa26(nim, nama, kelas, ipk);

            list.tambah(m);

        }
```

C. Mengurutkan Data Mahasiswa Berdasarkan IPK (Selection Sort)

1. Penambahan method selectionSort() di Kode Program Class MahasiswaBerprestasi26:

```
    void selectionSort() {
        for (int i=0; i < listMhs.length; i++) {
            int idxMin = 1;
            for (int j = i+1; j<listMhs.length; j++) {
                if (listMhs[j].ipk<listMhs[idxMin].ipk) {
                    idxMin = j;
                }
            }
            Mahasiswa26 tmp = listMhs[idxMin];
            listMhs[idxMin] = listMhs[i];
            listMhs[i]=tmp;
        }
    }
```

2. Penambahan di Kode Program Main MahasiswaDemo26 :

```
System.out.println(x: "Data yang sudah terurut menggunakan Selection Sort (ASC) : ");
list.selectionSort();
list.tampil();
```

3. Input dan Output Kode Program :
   - Input

```
Masukkan Data Mahasiswa ke-1
NIM    : 123
Nama   : Ali
Kelas  : 2B
IPK    : 3.9
Masukkan Data Mahasiswa ke-2
NIM    : 124
Nama   : ila
Kelas  : 2B
IPK    : 3.1
Masukkan Data Mahasiswa ke-3
NIM    : 125
Nama   : agus
Kelas  : 2B
IPK    : 3.6
Masukkan Data Mahasiswa ke-4
NIM    : 126
Nama   : tika
Kelas  : 2B
IPK    : 3.3
Masukkan Data Mahasiswa ke-5
NIM    : 127
Nama   : Udin
Kelas  : 2B
IPK    : 3.2
```

   - Output

```
Data yang sudah terurut menggunakan Selection Sort (ASC) :
Nama : ila
NIM : 124
Kelas : 2B
IPK : 3.1
-----------------------------
Nama : Ali
NIM : 123
Kelas : 2B
IPK : 3.9
-----------------------------
Nama : Udin
NIM : 127
Kelas : 2B
IPK : 3.2
-----------------------------
Nama : agus
NIM : 125
Kelas : 2B
IPK : 3.6
-----------------------------
Nama : tika
NIM : 126
Kelas : 2B
IPK : 3.3
-----------------------------
```

4. Jawaban dari pertanyaan :

Proses tersebut bertujuan untuk mengurutkan ipk dari yang paling tinggi ke paling rendah. Perulangan yang berada di dalam bertujuan untuk mencari nilai yang paling kecil, menggunakan perbandingan dengan nilai indeks paling awal. Sementara proses perulangan yang berada di luar bertujuan untuk mengulangi perulangan tersebut jika sudah ketemu angka yang paling kecil.

D. Mengurutkan Data Mahasiswa Berdasarkan IPK Menggunakan Insertion Sort

1. Penambahan kode di Class MahasiswaBerpestasi26 :

```
    void insertionSort() {
        for (int i = 0; i < listMhs.length; i++) {
            Mahasiswa26 temp = listMhs[i];
            int j = i;
            while (j>0 && listMhs[j-1].ipk > temp.ipk) {
                listMhs[j]=listMhs[j-1];
                j--;
            }
            listMhs[j]=temp;
        }
    }
}
```

2. Penambahan kode di Main MahasiswaDemo26 :

```
System.out.println(x: "Data yang sudah terurut menggunakan Insertion Sort (ASC) : ");
list.insertionSort();
list.tampil();

sc2.close();
```

3. Output kode program :

- Input

```
Masukkan Data Mahasiswa ke-1
NIM    : 111
Nama   : ayu
Kelas  : 2c
IPK    : 3.7
Masukkan Data Mahasiswa ke-2
NIM    : 22
Nama   : dika
Kelas  : 2c
IPK    : 3.0
Masukkan Data Mahasiswa ke-3
NIM    : 333
Nama   : ayu
Kelas  : 2c
IPK    : 3.8
Masukkan Data Mahasiswa ke-4
NIM    : 444
Nama   : susi
Kelas  : 2c
IPK    : 3.1
Masukkan Data Mahasiswa ke-5
NIM    : 555
Nama   : yayuk
Kelas  : 2c
IPK    : 3.4
```

- Output

```
Data yang sudah terurut menggunakan Insertion Sort (ASC) :
Nama : dika
NIM : 22
Kelas : 2c
IPK : 3.0
-----------------------------
Nama : susi
NIM : 444
Kelas : 2c
IPK : 3.1
-----------------------------
Nama : yayuk
NIM : 555
Kelas : 2c
IPK : 3.4
-----------------------------
Nama : ayu
NIM : 111
Kelas : 2c
IPK : 3.7
-----------------------------
Nama : ayu
NIM : 333
Kelas : 2c
IPK : 3.8
-----------------------------
```

4. Modifikasi Insertion Sort menjadi descending

```java
    void insertionSort() {
        for (int i = 0; i < listMhs.length; i++) {
            Mahasiswa26 temp = listMhs[i];
            int j = i;
            while (j>0 && listMhs[j-1].ipk < temp.ipk) {
                listMhs[j]=listMhs[j-1];
                j--;
            }
            listMhs[j]=temp;
        }
    }
}
```

Modifikasi hanya terdapat pada tanda besar dan kecil pada kondisi while.

- Output

```
Data yang sudah terurut menggunakan Insertion Sort (ASC) :
Nama : Sita
NIM : 126
Kelas : 2A
IPK : 3.9
-----------------------------
Nama : Miki
NIM : 127
Kelas : 2A
IPK : 3.7
-----------------------------
Nama : Ayu
NIM : 124
Kelas : 2A
IPK : 3.5
-----------------------------
Nama : Zidan
NIM : 123
Kelas : 2A
IPK : 3.2
-----------------------------
Nama : Sofi
NIM : 125
Kelas : 2A
IPK : 3.1
-----------------------------
```